

Striver SDE Sheet — Arrays Part I

1. Set Matrix Zeroes

Problem

Given an $m \times n$ matrix, whenever an element is `0`, make its entire row and column `0`.

Approach 1: Brute Force — Mark using a special value

Idea

Whenever `matrix[i][j] == 0`:

- Traverse its complete row.
- Traverse its complete column.
- Mark non-zero elements using a temporary value such as `-1`.
- In another traversal, convert all marked values to `0`.

Important

This works only when the temporary value cannot already appear in the matrix.

Complexity

- Time: $O(m \times n \times (m + n))$
 - Space: $O(1)$
-

Approach 2: Better — Separate row and column arrays

Idea

Create:

```
row[m]
col[n]
```

During the first traversal:

```
if matrix[i][j] == 0:
    row[i] = 1
    col[j] = 1
```

During the second traversal:

```
if row[i] == 1 or col[j] == 1:
    matrix[i][j] = 0
```

Complexity

- Time: $O(m \times n)$
- Space: $O(m + n)$

Approach 3: Optimal — Use first row and first column as markers

Core idea

Instead of separate marker arrays:

- `matrix[i][0]` marks whether row `i` must become zero.
- `matrix[0][j]` marks whether column `j` must become zero.
- Use one extra variable `col0` for the first column.

The first cell `matrix[0][0]` represents the first row, so it cannot also safely represent the first column.

Algorithm

```
col0 = 1

for every matrix[i][j]:
    if matrix[i][j] == 0:
        matrix[i][0] = 0

    if j == 0:
        col0 = 0
    else:
        matrix[0][j] = 0
```

Update the inner matrix in reverse:

```

for i from m - 1 to 0:
    for j from n - 1 to 1:
        if matrix[i][0] == 0 or matrix[0][j] == 0:
            matrix[i][j] = 0

    if col0 == 0:
        matrix[i][0] = 0

```

Why traverse in reverse?

If the first row or column is modified too early, marker information may be destroyed.

Complexity

- Time: $O(m \times n)$
- Extra space: $O(1)$

Revision key

First row stores column markers, first column stores row markers, and `col0` handles the first column.

2. Pascal's Triangle

Each element is generated using:

```

triangle[row][col]
=
triangle[row - 1][col - 1]
+
triangle[row - 1][col]

```

Boundary elements are always `1`.

There are usually three variants.

Variant 1: Find the element at row `r`, column `c`

Using 1-based indexing:

```
Element = C(r - 1, c - 1)
```

Combination formula:

```
nCr = n! / (r! * (n-r)!)
```

Calculating factorials directly can overflow and is inefficient.

Optimal calculation

```
result = 1

for i from 0 to r - 1:
    result = result * (n - i)
    result = result / (i + 1)
```

Here:

```
n = row - 1
r = column - 1
```

Complexity

- Time: $O(c)$
- Space: $O(1)$

Variant 2: Print a particular row

Brute force

Calculate every element using nCr .

For row r :

```
C(r-1, 0), C(r-1, 1), ..., C(r-1, r-1)
```

Complexity

- Time: approximately $O(r^2)$
- Space: $O(1)$ excluding output

Optimal row generation

Generate the next value from the previous value:

```
answer = 1
print answer

for col from 1 to row - 1:
    answer = answer × (row - col)
    answer = answer / col
    print answer
```

Example for row 5:

```
1 4 6 4 1
```

Complexity

- Time: $O(r)$
- Space: $O(1)$ excluding output

Variant 3: Generate the complete triangle

Generate every row using the optimal row-generation method.

```
for row from 1 to n:
    generateRow(row)
```

Complexity

- Time: $O(n^2)$
- Space: $O(n^2)$ for storing the answer

Revision key

Pascal element at (r, c) is $(r-1)C(c-1)$. Generate each row using the previous element instead of factorials.

3. Next Permutation

Problem

Find the next lexicographically greater permutation.

Example:

[1, 2, 3] → [1, 3, 2]

If no greater permutation exists:

[3, 2, 1] → [1, 2, 3]

Approach 1: Brute Force — Generate all permutations

Idea

1. Generate all permutations.
2. Sort them.
3. Find the current permutation.
4. Return the next permutation.

Complexity

- Time: approximately $O(n! \times n)$
- Space: $O(n! \times n)$

This is impractical.

Approach 2: Better — Built-in next permutation

Some languages provide a built-in function.

This is useful in practice but does not explain the algorithm expected in interviews.

Approach 3: Optimal

Observation

Consider:

```
[2, 1, 5, 4, 3, 0, 0]
```

From the right side, the sequence:

```
5, 4, 3, 0, 0
```

is decreasing.

No larger permutation can be formed by rearranging only this suffix.

We must modify the element just before it.

Step 1: Find the break point

Find the first index from the right where:

```
arr[i] < arr[i + 1]
```

```
index = -1

for i from n - 2 down to 0:
    if arr[i] < arr[i + 1]:
        index = i
        break
```

If no break point exists, the array is the largest permutation.

```
reverse entire array
```

Step 2: Find the smallest greater element

From the right, find the first element greater than `arr[index]`.

```
for i from n - 1 down to index + 1:
    if arr[i] > arr[index]:
        swap(arr[i], arr[index])
        break
```

Because the suffix is decreasing, the first greater element from the right is the smallest suitable element.

Step 3: Reverse the suffix

```
reverse(index + 1, n - 1)
```

This converts the suffix into its smallest possible order.

Example

Array: [2, 1, 5, 4, 3, 0, 0]

Break point:

```
1 < 5  
index = 1
```

Smallest element greater than **1** from the right:

3

Swap:

[2, 3, 5, 4, 1, 0, 0]

Reverse suffix:

[2, 3, 0, 0, 1, 4, 5]

Complexity

- Time: $O(n)$
- Space: $O(1)$

Revision key

Find dip → swap with the smallest greater element → reverse the suffix.

4. Maximum Subarray Sum — Kadane's Algorithm

Problem

Find the maximum sum among all contiguous subarrays.

Example:

`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

Maximum subarray:

`[4, -1, 2, 1]`

Maximum sum:

6

Approach 1: Brute Force — Generate all subarrays

Use three loops:

```
for i:
  for j:
    sum = 0
    for k from i to j:
      sum += arr[k]
```

Complexity

- Time: $O(n^3)$
- Space: $O(1)$

Approach 2: Better — Remove the third loop

For every starting index, keep extending the subarray.

```

maximum = -infinity

for i from 0 to n - 1:
    sum = 0

    for j from i to n - 1:
        sum += arr[j]
        maximum = max(maximum, sum)

```

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

Approach 3: Optimal — Kadane's Algorithm

Core idea

A negative running sum cannot help any future subarray.

Therefore:

- Add the current element.
- Update the maximum.
- Reset the running sum to 0 when it becomes negative.

```

sum = 0
maximum = -infinity

for value in array:
    sum += value
    maximum = max(maximum, sum)

    if sum < 0:
        sum = 0

```

Why update maximum before resetting?

For an all-negative array:

```
[-3, -2, -5]
```

The correct answer is -2 , not 0 .

Printing the maximum-sum subarray

Track:

```
start
answerStart
answerEnd
```

```
sum = 0
maximum = -infinity
start = 0

for i from 0 to n - 1:

    if sum == 0:
        start = i

    sum += arr[i]

    if sum > maximum:
        maximum = sum
        answerStart = start
        answerEnd = i

    if sum < 0:
        sum = 0
```

Complexity

- Time: $O(n)$
- Space: $O(1)$

Revision key

Carry the current sum only while it is useful. Drop it when it becomes negative.

5. Sort an Array of 0s, 1s and 2s

Also called the **Dutch National Flag Algorithm**.

Approach 1: Brute Force — Sorting

Use a standard sorting algorithm.

```
sort(array)
```

Complexity

- Time: $O(n \log n)$
 - Space: depends on the sorting algorithm
-

Approach 2: Better — Counting

Count the occurrences of:

```
count0  
count1  
count2
```

Overwrite the array:

```
first count0 positions → 0  
next count1 positions  → 1  
remaining positions    → 2
```

Complexity

- Time: $O(2n) \rightarrow O(n)$
- Space: $O(1)$

This requires two passes.

Approach 3: Optimal — Dutch National Flag

Maintain three pointers:

```
low  
mid  
high
```

Array regions:

```
[0 ... low-1]      → 0  
[low ... mid-1]    → 1  
[mid ... high]     → unsorted  
[high+1 ... n-1]   → 2
```

Initially:

```
low = 0  
mid = 0  
high = n - 1
```

Cases

Case 1: `arr[mid] == 0`

Move zero to the left.

```
swap(arr[low], arr[mid])  
low++  
mid++
```

Case 2: `arr[mid] == 1`

One is already in the correct middle region.

```
mid++
```

Case 3: `arr[mid] == 2`

Move two to the right.

```
swap(arr[mid], arr[high])  
high--
```

Do not increment `mid` here because the element received from `high` has not been checked.

Complete logic

```
while mid <= high:

    if arr[mid] == 0:
        swap(arr[low], arr[mid])
        low++
        mid++

    else if arr[mid] == 1:
        mid++

    else:
        swap(arr[mid], arr[high])
        high--
```

Complexity

- Time: $O(n)$
- Space: $O(1)$

Revision key

Zero goes left, one stays in the middle, two goes right.

6. Stock Buy and Sell

Problem

Given stock prices on different days, find the maximum profit from exactly one buy and one sell.

The stock must be bought before it is sold.

Example:

Prices: [7,1,5,3,6,4]

Best transaction:

```
Buy at 1  
Sell at 6  
Profit = 5
```

Approach 1: Brute Force

Try every buying day and every later selling day.

```
maximumProfit = 0  
  
for buy from 0 to n - 1:  
    for sell from buy + 1 to n - 1:  
        profit = prices[sell] - prices[buy]  
        maximumProfit = max(maximumProfit, profit)
```

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

Approach 2: Optimal — Track minimum price

Core idea

For each selling day:

- Keep the minimum price seen before that day.
- Calculate the profit if sold today.
- Update the maximum profit.

```
minimumPrice = prices[0]  
maximumProfit = 0  
  
for i from 1 to n - 1:  
    profit = prices[i] - minimumPrice  
    maximumProfit = max(maximumProfit, profit)  
    minimumPrice = min(minimumPrice, prices[i])
```

Why does this work?

For every selling price, the best buying price is the smallest price that occurred before it.

Example

Prices: [7,1,5,3,6,4]

Day	Price	Minimum so far	Profit
0	7	7	-
1	1	1	0
2	5	1	4
3	3	1	2
4	6	1	5
5	4	1	3

Maximum profit:

5

Edge case

For continuously decreasing prices:

[7,6,4,3,1]

No profitable transaction exists, so return:

0

Complexity

- Time: $O(n)$
- Space: $O(1)$

Revision key

Treat each day as a selling day and remember the cheapest buying price seen before it.

Arrays Part I — Final Revision Table

Problem	Optimal technique	Time	Extra space
Set Matrix Zeroes	First row/column as markers	$O(mn)$	$O(1)$
Pascal's Triangle	Combination / previous element	$O(n^2)$	Output space
Next Permutation	Dip, swap, reverse	$O(n)$	$O(1)$
Maximum Subarray Sum	Kadane's algorithm	$O(n)$	$O(1)$
Sort 0s, 1s and 2s	Dutch National Flag	$O(n)$	$O(1)$
Stock Buy and Sell	Track minimum-so-far	$O(n)$	$O(1)$

One-Line Memory Tricks

Set Matrix Zeroes:

Use the first row and column as marker arrays.

Pascal's Triangle:

Element = $(row-1)C(column-1)$.

Next Permutation:

Find dip → swap → reverse suffix.

Kadane:

Reset the running sum when it becomes negative.

Sort 0,1,2:

Low for 0, mid for checking, high for 2.

Stock Buy/Sell:

Maintain minimum price and calculate today's profit.